DESIGN & ANALYSIS OF ALGORITHMS

PROJECT REPORT SUBMISSION FORM — EVIDENCE SHEET

Assignment: Tasks 1–6	**Course / section:** DAA / AIML Panthers	**Submitted on:** 13 Oct 2026
Project / framework title: Adaptive Network Optimization System		
Student: Kartikeya Parcha-2116 M.suchith-2144 L.Mohith-2137 K.Chinmai Datha-2126 C.Charith-1026 M.rohan-1251	**Team members / roll numbers:** Kartikeya Parcha-2116 M.suchith-2144 L.Mohith-2137 K.Chinmai Datha-2126 C.Charith-1026 M.rohan-1251	
Code / repository URL: https://github.com/suchith71/DAA-project	**Report / demo link:** https://github.com/suchith71/DAA-project	

PURPOSE Demonstrate how a real network-optimization problem is modelled, suitable syllabus algorithms are justified and compared, complexity and correctness are analyzed, implementations are tested, and algorithm choice adapts when conditions change.

1. PROBLEM ABSTRACTION & MODELLING

Scenario / decision to solve: Optimize a weighted network by minimizing connection and travel costs, maintaining searchable records, prioritizing maintenance events, and choosing upgrades under a limited budget.

Entities / inputs / size parameters: Vertices V, edges E, edge weights/costs, n records, n priority events, n upgrades, budget capacity W, and n maintenance sites.

Objective + constraints: Minimize MST and maintenance-tour costs; compute all-pairs shortest paths; maintain efficient records and priorities; maximize upgrade benefit without exceeding budget. TSP tour visits each required site once and returns to the start.

Representation: Weighted graph; optimization; state/search; dynamic programming; data structures (trees and heaps).

2. ALGORITHM CHOICE & FRAMEWORK DESIGN

Implemented DAA techniques planned: Prim, Kruskal, Floyd–Warshall, B-Tree, Red-Black Tree, Binomial Heap, Fibonacci Heap, Fractional Knapsack, 0/1 Knapsack, TSP Backtracking and/or Branch and Bound.

Alternatives compared: Prim vs Kruskal; B-Tree vs Red-Black Tree; Binomial vs Fibonacci Heap; Fractional vs 0/1 Knapsack; exact TSP search strategies.

Selection rule: For MST, consider graph density and measured runtimes; for record and priority structures, compare operation requirements and measured costs; for Knapsack, select based on whether items can be divided; for TSP, use exact search only at manageable sizes. Rules must be implemented and validated.

Workflow: Scenario → model → choose algorithm → execute → measure → validate → explain. Architecture diagram: include in the report.

3. COMPLEXITY & CORRECTNESS

Technique / operation	Expected time	Space	Assumption / correctness argument
Prim (binary heap)	O((V+E) log V)	O(V+E)	Connected, weighted, undirected graph; result has V−1 edges and no cycles.
Kruskal	O(E log E)	O(V+E)	Sort edges; Union-Find prevents cycles; compare MST total weight.
Floyd–Warshall	$O(V^3)$	$O(V^2)$	Distance matrix; negative cycles must be detected.
B-Tree / Red-Black Tree	O(log n) per operation*	O(n)	Balanced structure; verify ordering and balancing invariants.
Binomial / Fibonacci Heap	Operation-dependent	O(n)	Check heap-order property and priority results; Fibonacci bounds are amortized.
Fractional Knapsack	O(n log n)	O(n) typical	Greedy by value/weight ratio; fractional items allowed.
0/1 Knapsack DP	O(nW)	O(nW) full table	Items indivisible; pseudo-polynomial in capacity W.

Exact TSP	Factorial/exponential	Implementation-dependent	Optimal tour if exhaustive search/bounds are correct and algorithm completes.

Complexity status: Polynomial for MST, Floyd–Warshall, and balanced-tree operations; pseudo-polynomial for standard 0/1 Knapsack DP; NP-hard optimization for TSP; NP-complete decision version of TSP.

4. IMPLEMENTATION & VALIDATION

Language / tools: Python. Use modular files/functions for each algorithm, shared input formats where appropriate, and time.perf_counter() for elapsed-time measurement.

Correctness tests to run: Normal, boundary, large and difficult cases. Include disconnected graph; single/empty inputs where supported; repeated keys; empty heap; zero budget; items exceeding capacity; and small TSP instances. Record which tests actually passed after execution.

Code proof: Add the real repository or source-file page/reference. Do not claim tests passed until they have been run.

5. EXPERIMENTS & ADAPTATION

Input / condition	Size n	Algorithm / structure	Runtime / ops / memory	Result / observation
Baseline: sparse graph	Record V, E	Prim vs Kruskal	Measure on execution	Compare MST cost and runtime
Changed: dense graph / larger V	Record V, E	Prim vs Kruskal	Measure on execution	Check whether selection changes
Other modules	Record n and W	Compare paired algorithms	Measure relevant operations	Verify outputs and scalability

Decision / behaviour changed because: The implemented rule uses observed graph density, input size, operation requirements and/or measured runtime. Attach actual measurements or a figure/table; values are not filled here because execution results were not supplied.

6. EXPLANATION, LIMITS & DEMONSTRATION

Main finding and limitation: The best-performing algorithm depends on representation, input size and workload; asymptotic complexity does not guarantee the fastest runtime on every input. Floyd–Warshall uses $O(V^2)$ memory and $O(V^3)$ time; exact TSP can become infeasible as sites grow.

Theory versus observation: Compare measured runtime trends against complexity expectations; note machine, implementation, input distribution and timing noise.

NP-Hard / NP-Complete: Optimization TSP is NP-hard. Decision TSP asks whether a tour of cost at most B exists and is NP-complete; a proposed tour can be verified in polynomial time.

Submission checklist: [ ] integrated framework [ ] working code [ ] alternatives compared [ ] time/space analysis [ ] four test families [ ] adaptive decision [ ] report + demo

Important: Add your actual team/roll numbers, repository and demo links, and measured runtimes before submitting. This draft describes the intended framework; it does not certify that code has been executed or that tests have passed.